
Chickenhot

Manual de Despliegue

Sistema de delivery — Backend (FastAPI) · Web (React) · App móvil (Flutter)

Versión 1.0 · Junio 2026

1. Arquitectura del sistema

Todo el backend, la base de datos y la web corren en un mismo servidor Linux (Ubuntu en AWS EC2). La app móvil se distribuye aparte (APK/IPA) y consume la misma API.

Backend	FastAPI (Python) servido por uvicorn en el puerto 8000 , gestionado por systemd (servicio chikenhot).
Base de datos	PostgreSQL 18 (servicio postgresql@18-main), base chikenhot_db .
Servidor web	Nginx en el puerto 80 : sirve la web estática desde /var/www/html y hace proxy de /api/v1 -> uvicorn:8000 .
Frontend web	React + Vite. Se compila a dist/ y se copia a /var/www/html .
App móvil	Flutter (Android + iOS). APK/AAB para Android; IPA para iOS (vía Codemagic).
API base	http://<IP-o-dominio>/api/v1

La IP actual del servidor es **52.70.207.124**. **Atención:** si el servidor está en un **AWS Academy Learner Lab**, la IP CAMBIA cada vez que el lab se reinicia (~4 h). Ver la sección 7 para soluciones definitivas.

2. Estructura del repositorio

```
app_delivery/  
  backend/      API FastAPI (app/, requirements.txt, .env)  
  frontend/     Web React + Vite (src/, dist/, .env)  
  mobile/       App Flutter (lib/, android/, ios/, pubspec.yaml)  
  codemagic.yml CI en la nube para Android e iOS  
  chickenhot.apk / chickenhot.aab  artefactos Android (no en git)
```

El despliegue se hace por **git**: se sube el código a GitHub y en el servidor se hace **git pull**.

3. Requisitos previos

- Acceso por **SSH** al servidor (clave .pem del lab o del proveedor).
- En el servidor: Python 3 + venv, PostgreSQL, Nginx, Node.js 18+ y npm.
- En tu PC (para el móvil): Flutter 3.35+, Java 17, Android SDK.
- Git configurado y el repositorio en GitHub.

4. Despliegue del Backend (API)

4.1 Variable de entorno (una sola vez)

El archivo **backend/.env** NO se sube a git (está en .gitignore). En el servidor debe existir con la conexión a la base de datos:

```
# backend/.env (en el servidor)  
DATABASE_URL=postgresql://postgres:TU_PASSWORD@localhost:5432/chickenhot_db  
SECRET_KEY=clave-larga-y-secreta-para-produccion  
ALGORITHM=HS256  
ACCESS_TOKEN_EXPIRE_MINUTES=60  
REFRESH_TOKEN_EXPIRE_DAYS=7
```

4.2 Servicio systemd (una sola vez)

Hace que la API arranque sola con el servidor y se reinicie si falla:

```
sudo tee /etc/systemd/system/chickenhot.service > /dev/null << 'EOF'
[Unit]
Description=Chickenhot FastAPI Backend
After=network.target postgresql.service

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/app_delivery/backend
ExecStart=/home/ubuntu/app_delivery/backend/venv/bin/uvicorn app.main:app --host
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable chickenhot
sudo systemctl start chickenhot
```

Migraciones automáticas: al arrancar, el backend crea las tablas que falten y añade columnas nuevas (first_name, last_name, vehicle_photo, dni_photo, last_assigned_at) con **ADD COLUMN IF NOT EXISTS**. No necesitas ejecutar ALTER ni migraciones manuales.

5. Despliegue del Frontend web

5.1 Variable de entorno (una sola vez)

```
# frontend/.env (apunta a la API)
VITE_API_URL=http://52.70.207.124/api/v1
```

5.2 Compilar y publicar

```
cd ~/app_delivery/frontend
npm install          # solo la primera vez o si cambian dependencias
npm run build        # genera dist/
sudo cp -r dist/* /var/www/html/ # publica en Nginx
```

6. Flujo de despliegue (cada actualización)

Este es el procedimiento normal cada vez que hay cambios nuevos:

Paso 1 — En tu PC: subir los cambios

```
git push
```

Paso 2 — En el servidor (SSH): actualizar y aplicar

```
cd ~/app_delivery && git pull

# Backend (si cambió backend/):
sudo systemctl restart chickenhot

# Frontend (si cambió frontend/):
cd frontend && npm run build && sudo cp -r dist/* /var/www/html/
```

Paso 3 — En el navegador

Recargar con **Ctrl + Shift + R** (recarga forzada) para evitar el caché.

Regla práctica: si solo tocaste backend -> basta **restart chickenhot**. Si tocaste la web -> **build + copy**. Si tocaste el móvil -> recompilar el APK.

7. IP fija y hosting (importante)

Si el servidor está en un **AWS Academy Learner Lab**, hay límites que NO se pueden desactivar: la sesión dura ~4 h, la instancia se detiene sola y al reiniciar **la IP pública cambia**. Por eso la app deja de conectar y hay que reconfigurar la IP. Opciones:

DuckDNS (gratis)	Dominio gratis que se auto-actualiza a la IP nueva en cada arranque. La app apunta a http://tudominio.duckdns.org/api/v1 y nunca más se toca.
Oracle Always Free	VM gratis para siempre , 24/7, con IP pública fija. La mejor opción para producción sin costo.
AWS Free Tier	Cuenta AWS normal (no Academy): t3.micro gratis 12 meses, 24/7, con Elastic IP real que no cambia.

Recomendación: para uso real, migrar a Oracle Always Free o AWS Free Tier; el stack (FastAPI + Postgres + Nginx) se instala igual que en las secciones 4 y 5.

8. App móvil (Android e iOS)

8.1 Configurar la API

La URL del backend está en **mobile/lib/core/config.dart**:

```
static const String _defaultBaseUrl = 'http://52.70.207.124/api/v1';
```

8.2 Compilar Android (en tu PC)

```
cd mobile
flutter pub get
flutter build apk --release # -> build/app/outputs/flutter-apk/app-release.apk
flutter build appbundle --release # -> .../bundle/release/app-release.aab (Play Store)
```

El APK ya compilado está en la raíz del proyecto como **chickenhot.apk**. Se instala directo en el celular (activar 'orígenes desconocidos').

8.3 Compilar iOS (Codemagic, en la nube)

No se puede compilar iOS en Windows. El repo incluye **codemagic.yaml** con los workflows **Chickenhot Android** y **Chickenhot iOS**. Pasos:

- Entra a codemagic.io, conecta el repositorio de GitHub y elige **Switch to YAML configuration**.
- Para iOS necesitas cuenta **Apple Developer** (\$99/año) y crear una **API key de App Store Connect** nombrada **CodemagicAppStoreKey**.
- El bundle id está fijado a **com.chickenhot.chickenhot**.
- Lanza el workflow -> recibes el .ipa por correo. (El de Android sale sin firma de pago.)

9. Datos y credenciales de prueba

Cuentas creadas por el seed inicial (existen en la base de datos; los botones de acceso rápido se quitaron del login por seguridad):

Admin	admin@chickenhot.pe / admin123
Cliente	cliente@chickenhot.pe / cliente123
Repartidor	delivery@chickenhot.pe / delivery123

En producción real, cambia estas contraseñas y el **SECRET_KEY** del .env.

10. Comandos útiles y diagnóstico

Estado de los servicios

```
sudo systemctl status chickenhot      # API
sudo systemctl status nginx           # web / proxy
sudo systemctl status postgresql@18-main # base de datos
```

Logs del backend

```
journalctl -u chickenhot -n 80 --no-pager
```

Probar la API localmente en el servidor

```
curl -sL http://localhost:8000/api/v1/products/ | head -c 300
```

Problemas frecuentes

La app no conecta	La IP del lab cambió. Revisa la IP nueva (AWS Details) y actualiza frontend/.env y config.dart, o usa DuckDNS (sección 7).
La web no actualiza	Falta copiar dist/ a /var/www/html, o caché del navegador. Rebuild + Ctrl+Shift+R.
Error 500 al crear pedido	Reinicia el backend para aplicar la auto-migración (columnas nuevas).
La API responde 307	Es normal: FastAPI redirige /ruta/ -> /ruta. El cliente lo sigue solo.